

Volatility-Adjusted Memory Protection

A causal control knob for continual learning and LLM agent memory

Richard Emate

richard@theemate.com · Independent · 16 August 2026

doi.org/10.5281/zenodo.21962419

Preprint. Results from VoltMem 0.2.x (git tag v0.2.2). The current package (0.4.0) changes the default write law and is not claimed here.

Abstract

Standard continual-learning methods such as Elastic Weight Consolidation apply a uniform protection strength to all past knowledge, forcing a stability–plasticity tradeoff. We scale protection per domain by measured volatility *before* any update, and validate claims with negative controls: shuffling or inverting the domain→volatility map must degrade performance monotonically (REAL > SHUFFLE > SWAP). On a synthetic benchmark with disjoint task inputs, both retention and adaptation improve simultaneously. On Split-MNIST the honest result is subtler: volatility weighting is not a free-lunch Pareto win but a **causal control knob** that steers the tradeoff. The same principle powers VoltMem, an open-source Python memory library for LLM agents. On scripted multi-turn scenarios, volatility-aware write and retrieval beat flat and inverted controls (balanced score 0.597, only policy strong on both stable and volatile axes). On a noisy retrieval haystack the win is staleness resistance (0% stale@1 vs 20% cosine-only; separation 0.153 vs −0.003), not top-1 hit rate. A scripted case study vs Mem0 is 3/3 on current-truth for mood, preference, and location updates. On LongMemEval-S (n=60, chunk-calibrated RAG ingest), retrieval answer@5 is 70.0% for VoltMem—tying plain cosine and beating the inverted-volatility control (66.7%)—without claiming public-benchmark SOTA (uniform-volatility ablation reaches 71.7%). We report limitations, embedding-backend variance, failed early wins discarded by sabotage controls, and open gaps in automatic domain discovery.

1. Introduction

Continual-learning systems and LLM agent memory layers both face the *stability–plasticity dilemma*: protect old knowledge too strongly and the system cannot adapt; too weakly and it forgets or goes stale. Elastic Weight Consolidation (EWC) [1] penalises parameter changes in proportion to Fisher information, but applies one global protection strength uniformly. Agent memory systems such as Mem0 [6] similarly store relevant facts without modelling how fast different kinds of knowledge age. Not all knowledge changes at the same rate: biographical facts are stable; mood, location, and current tasks are volatile.

This work makes three contributions:

- **Per-domain volatility scaling** of memory protection and retrieval freshness, with volatility measured *before* any update to avoid circularity.
- **Sabotage negative controls** (REAL > SHUFFLE > SWAP) that gate every non-synthetic empirical claim.
- **VoltMem**, an open-source Python memory library validating the same principle on agent write policy, retrieval ranking, and scripted comparisons against Mem0 [6] and LongMemEval [5].

Section 2 states the hypothesis and core equations. Section 3 describes benchmarks and controls. Section 4 records discarded early wins. Section 5 reports results. Section 6 lists limitations.

Related work

Continual learning methods range from regularisation (EWC [1], synaptic intelligence [3]) to replay and constrained optimisation (GEM [2], progress & compress [4]). Surveys [7] frame the stability–plasticity tradeoff as the central challenge. LongMemEval [5] benchmarks long-horizon memory retrieval in chat assistants; Mem0 [6] targets production agent memory with vector storage and consolidation. VoltMem differs by assigning

explicit volatility priors per domain and validating behaviour under inverted controls, rather than claiming uniform benchmark dominance.

2. Hypothesis

Scale the EWC penalty per stored task by how volatile that task's domain actually is, measured before any protection is applied. High measured volatility → weak protection; low volatility → strong protection.

Protection weight

$$w_d = 1 / V_d^\gamma$$

The same volatility prior drives the open-source VoltMem memory layer. Write path: an escalation score versus a volatility-scaled threshold; retrieve path: down-rank stale volatile chunks.

Escalation score and threshold

$$E_t = [(M_t \cdot R_t) / C^\alpha] \cdot V_d \cdot G_t$$

$$\theta_t = \theta_0 \cdot (1 / V_d) \cdot L_t \quad (\text{audit iff } E_t > \theta_t)$$

Retrieval freshness

$$\text{staleness} = 1 - \exp(-V_d \cdot \text{age}_{\text{days}})$$

$$\text{score} = \text{similarity} \cdot (1 - V_d \cdot \text{staleness})$$

The original prediction was that this improves both retention and adaptability *simultaneously*. That turned out to be true only in a specific regime; the general, real-data claim is weaker and more precise (Section 5).

3. Method

Synthetic benchmark

12 sequential binary classification tasks on 2D Gaussian blobs, alternating between two domain types sharing a single trunk and single output head — genuine weight competition, no dedicated capacity to hide in. **Stable domain:** blob centres jitter ± 0.15 (boundary barely moves). **Volatile domain:** blob centres relocate substantially each task.

Real data — Split-MNIST

To answer the obvious objection (“2D blobs are a toy”), the same idea runs on Split-MNIST. A stable pool of digits keeps a fixed digit→class mapping every task; a disjoint volatile pool is randomly re-partitioned each task so successive volatile tasks contradict each other. Both feed one shared trunk and one 2-way head, so they genuinely compete for weights (real forgetting), but because the pools are disjoint, protecting stable features does not directly fight volatile inputs.

Volatility estimation (pre-update) & penalty

Before any gradient step on a new task, measure the model's loss using only knowledge from prior tasks — a clean mismatch signal uncorrupted by the EWC penalty — and track a running EMA per domain. Each stored task's EWC penalty is then weighted by w_d above: high $V_d \rightarrow$ small $w_d \rightarrow$ weak protection; low $V_d \rightarrow$ large $w_d \rightarrow$ strong protection.

Negative control (the load-bearing part)

Every non-synthetic claim is gated on a **sabotage / negative control**: re-run the identical pipeline with the domain→volatility mapping (i) shuffled and (ii) inverted. A genuine effect must degrade monotonically **REAL** > **SHUFFLE** > **SWAP**. If a “win” survives shuffling or inversion, it is generic non-uniform regularisation, not the

volatility signal, and it is discarded. Several early wins failed this test — that filter is why the surviving claims are trustworthy.

4. Failures Worth Keeping

Early attempts produced flat ties or artifacts — kept because diagnosing them was most of the actual work:

- **Separate output heads** — no genuine weight competition, nothing to forget in either method.
- **Volatility measured post-update** — contaminated by EWC clamping; it tracked the intervention, not the domain.
- **Models trained to 100% confidence** — collapses Fisher information toward zero, making the penalty inert. Fixed by label smoothing and fewer epochs.
- **High-capacity MNIST “win”** (hidden=512, +7pp retention) — *failed the negative control*: shuffling/inverting the pairing kept most of the gain, so it was generic regularisation, not volatility. Discarded as a headline result.
- **Capacity-saving hypothesis** (a small volatility model matching a larger baseline) — *falsified* by the sweep (Section 5.3).

5. Results

5.1 Synthetic — both axes improve (regime-specific)

Metric	Baseline EWC	Volatility-Adjusted
Early-task stable retention	0.935	0.962 (+2.7pp)
Late-task volatile adaptation	0.576	0.628 (+5.2pp)
Avg accuracy — stable tasks	0.936	0.963 (+2.7pp)
Avg accuracy — volatile tasks	0.617	0.633 (+1.6pp)

8 runs, 12 tasks each. The model inferred the volatile domain was $\sim 2.4\times$ more unstable than the stable one, purely from pre-update loss. Holds when tasks use disjoint inputs over a shared trunk; not a universal Pareto win (see 5.2).

5.2 Split-MNIST — a causal control knob

In a genuinely-competing regime (hidden=128, $\lambda \approx 300$, 16 tasks, 6 runs) the effect *passes* the negative control. Reported as a stability index = retention – adaptation:

	baseline	REAL	SHUFFLE	SWAP
stability index	0.317	0.460	0.457	0.405

REAL – SWAP = +0.055 in the predicted direction: inverting which domains are “volatile” reliably tilts the model toward plasticity. The volatility signal causally steers the tradeoff — but it trades adaptation for retention along a frontier a well-tuned uniform baseline could also reach.

5.3 Capacity — robustness to under-tuned protection

The parameter-savings hypothesis was falsified. The favourable, causal benefit appears in **under-protected large models**, where uniform EWC catastrophically forgets and volatility rescues the stable domain (hidden=256, $\lambda \approx 300$, 4 runs):

- retention: baseline 0.877 $\rightarrow$ **0.954 (+7.7pp)**; adaptation only -2.9pp ,
- passes the sabotage control (shuffle/swap keep only +1.3pp).

Interpretation: volatility auto-allocates a fixed protection budget to stable knowledge, reducing the need to hand-tune λ per capacity. A well-tuned uniform baseline can match it; volatility mainly reduces tuning burden.

5.4 LLM-memory library — volatility-driven under control

The same principle powers a small persistent-memory library. Running it end-to-end under three volatility profiles — real (priors), flat (all equal), swap (inverted):

Capability	real	flat	swap
Selective updating (accuracy)	100%	83%	50%
Freshness-aware retrieval (separation)	+0.589	+0.202	-0.267

The swap control degrading below flat is the causal evidence: flipping which domains are volatile flips behaviour in the wrong direction.

5.5 Reliability-weighted volatility EMA

A multi-turn robustness fix: the volatility EMA's learning rate is scaled by source reliability:

$$\alpha = (1 - \beta) \cdot \text{clamp}(R_t, 0, 1)$$

$$V \leftarrow (1 - \alpha) \cdot V + \alpha \cdot M_t$$

A low-trust observation moves a stable memory **2.5× less** than the prior logic; reliable-source updates are unchanged (backward compatible). A sustained weak stream crosses $V \geq 0.5$ at turn 8 versus the old logic's turn 2, and stable content is never wrongly overwritten. (Also fixed: `observe()` previously updated the EMA twice per call.) Honest limit: reliability scales the step size, not the EMA's fixed point, so persistent contradictions still raise volatility over time — arguably correct, but not a hard “never drifts” guarantee.

5.6 LLM-agent memory — beating naive policies on both failure modes

As memory for an LLM agent, the same principle is tested over many noisy sessions where facts drift at domain-appropriate rates. A memory layer can fail two ways at once: go *stale* on volatile facts, or get *corrupted* by a confident-but-wrong observation on stable facts. Naive policies are forced onto one side; VoltMem escapes the tradeoff (24 sessions × 20 runs; balanced = harmonic mean of stable & volatile accuracy):

system	overall	stable	volatile	balanced
voltmem (real)	0.522	0.578	0.617	0.597
voltmem (flat)	0.361	0.578	0.188	0.283
voltmem (swap)	0.347	0.407	0.194	0.263
always-overwrite	0.573	0.360	0.767	0.490
never-overwrite	0.361	0.578	0.188	0.283
reliability-threshold	0.579	0.437	0.548	0.486

VoltMem-real is the only policy strong on both axes; each naive policy catastrophically fails one, and a pure reliability rule is mediocre on both. real > flat > swap is the causal control. Building the demo also surfaced a real bug: `observe()` updated the volatility EMA before the escalation decision, so one confident blip inflated volatility and lowered its own threshold, overwriting a stable fact on first contact. Fixed by deciding against the volatility known before the observation, then learning from it.

5.7 Slot-aware linking — write-path wedge vs Mem0

Embedding-based `remember()` must associate a new turn with an existing memory before `observe()` can apply the escalation rule. A single global similarity threshold fails on paraphrases (MiniLM scores 0.44–0.53 on mood/preference rephrasings). **Slot-aware linking** adds a domain-scoped fallback: volatile singleton slots (mood, location, task) and preference sibling domains link at a volatility-scaled lower threshold.

scenario	VoltMem	Mem0 (real)
mood update	1 fact, correct top	2 facts, stale top
preference change	1 fact, correct top	adopts blip (partial)

location change	1 fact, correct top	2 facts, stale top
-----------------	----------------------------	--------------------

Scripted case study (VoltMem 3/3 current-truth) vs Mem0 [6] (open-source, gpt-4o-mini, text-embedding-3-small). Mem0 is stale-top on mood and location, and adopts the preference blip. Not a public-benchmark SOTA claim. Embeddings: MiniLM [8].

5.8 Retrieval haystack & LongMemEval-S

Noisy haystack (5 slots $\times$ 20 runs, 6 stale decoys + 3 distractors per slot, top-5):

system	current@1	current@5	stale@1	separation
voltmem_real	0.600	1.000	0.000	0.153
voltmem_flat	0.800	1.000	0.200	0.133
voltmem_swap	0.650	1.000	0.200	0.111
similarity_only	0.600	0.800	0.200	−0.003

PASS: real avoids stale volatile traps (0% stale@1 vs 20% cosine), finds current in top-5 more often (100% vs 80%), separation real > swap. current@1 is not above flat (0.600 vs 0.800) — the win is staleness resistance, not raw top-1 hit rate.

LongMemEval-S (n=60, chunk-calibrated ingest: user $\rightarrow$ *stated_preference*, assistant $\rightarrow$ *opinion*):

system	answer@5	evidence@5
voltmem_flat	0.717	0.850
voltmem_real	0.700	0.833
similarity_only	0.700	0.850
voltmem_swap	0.667	0.817

question type	real	flat	swap	cosine
single-session-preference	0.700	0.700	0.600	0.700
single-session-assistant	0.900	0.900	0.900	0.900
single-session-user	0.800	0.900	0.700	0.900
knowledge-update	0.600	0.600	0.500	0.600
temporal-reasoning	0.600	0.600	0.700	0.600
multi-session	0.600	0.600	0.600	0.500

Real ties cosine overall (70.0%); flat leads slightly (71.7%); swap (66.7%) < real. Preference type recovers to 0.700 (was 0.300 with heuristic per-turn domains). Not a public-benchmark SOTA claim.

6. Caveats

- **Not a free-lunch accuracy method.** On real data it is a control knob / tuning-robustness tool, not a universal Pareto improvement.
- **Regime-dependent.** The “improve both axes” result needs disjoint task inputs over a shared trunk; benefits are strongest in under-protected large models.
- **Small-scale benchmarks only.** 2D blobs and Split-MNIST; Split-CIFAR, larger nets, and longer task streams are pending.
- **EMA fixed-point drift.** Reliability weighting slows erosion but does not anchor the estimate to the domain prior; a prior-anchored update is the natural next step.
- **Embedding backend variance.** Linking thresholds are calibrated for sentence-transformers (MiniLM); production should pin or calibrate per backend.

- **LongMemEval overall.** At $n=60$ (chunk-calibrated ingest), real ties cosine (70.0%); flat leads (71.7%); swap (66.7%) < real.
- **Manual partitioning.** Automatic volatility detection from gradient-conflict signals remains open work.
- **Software snapshot.** Library tables use VoltMem 0.2.x (git tag v0.2.2). `pip install voltmem` currently resolves to 0.4.0, whose default write law is not evaluated here.
- **Vector index (v0.2).** SQLite ANN accelerates candidate retrieval; volatility re-rank is unchanged (engineering, not a separate claim).
- **No replay-baseline comparison** yet (GEM [2], A-GEM, synaptic intelligence [3]) — volatility-weighting may be additive or redundant.

7. Conclusion

Volatility-adjusted memory protection is a validated **control knob** on the stability–plasticity tradeoff, not a free-lunch accuracy method. Negative controls (REAL > SHUFFLE > SWAP) separate genuine volatility-driven behaviour from generic non-uniform regularisation. The same principle transfers to LLM agent memory in VoltMem: selective updates, freshness-aware retrieval, and current-truth behaviour in scripted scenarios where uniform memory layers retain stale facts.

Future work: prior-anchored volatility estimates, automatic domain discovery from gradient conflict, replay baselines, and larger-scale continual-learning benchmarks. Code, reproduction scripts, and extended tables: github.com/Rouche01/voltmem.

References

- [1] Kirkpatrick, J., et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.
- [2] Lopez-Paz, D., & Ranzato, M. Gradient episodic memory for continual learning. *NeurIPS*, 2017.
- [3] Zenke, F., et al. Continual learning through synaptic intelligence. *ICML*, 2017.
- [4] Schwarz, J., et al. Progress & compress: A scalable framework for continual learning. *ICML*, 2018.
- [5] Wu, Z., et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *ICLR*, 2025.
- [6] Chhikara, P., et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv:2504.19413*, 2025.
- [7] Parisi, G. I., et al. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.
- [8] Reimers, N., & Gurevych, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP*, 2019.

DOI: <https://doi.org/10.5281/zenodo.21962419> · Code: github.com/Rouche01/voltmem · Reproduce: `git checkout v0.2.2` · PyPI latest is 0.4.0 (different default write law)